
Paradigmas de Programación

— —

CYPLP

Paradigmas

Un paradigma de programación es un **estilo de desarrollo de programas**, un modelo para resolver problemas computacionales. Los lenguajes de programación, necesariamente, se encuadran en uno o varios paradigmas a la vez, a partir del tipo de órdenes que permiten implementar, tiene una **relación directa con su sintaxis**.

```
1 procedure DandC(pbm, sol)
2 local var aux;
3 begin
4   if easy(pbm) then
5     solve(pbm)
6   else
7     begin
8       divide(pbm, subpbm, aux);
9       parallel i in 1..numP
10         DandC(subpbm[i], subsol[i]);
11       combine(subsol, aux, sol);
12     end
13 end;
```



```
?- p_exam(noest,notar).
false.
```

```
?- p_exam(est,tar).
true .
```

```
?- p_exam(noest,tar).
true.
```

```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

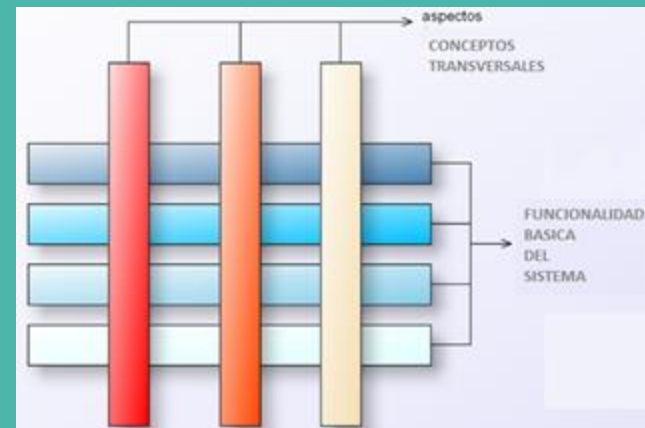
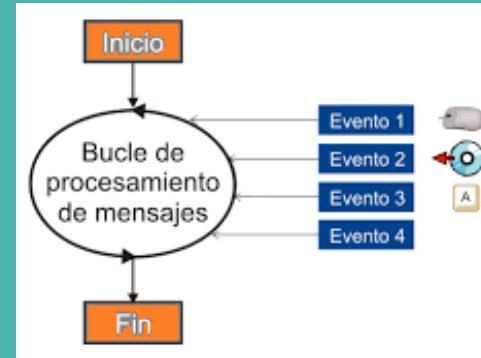
Principales Paradigmas

- **Imperativo:** sentencias + secuencias de comandos
- **Declarativo.** Los programas describen los resultados esperados sin listar explícitamente los pasos a llevar a cabo para alcanzarlos.
 - **Lógico.** Aserciones lógicas: hechos + reglas, es declarativo
- **Funcional.** Los programas se componen de funciones
- **Orientado a Objetos :** Métodos + mensajes.

Principales Paradigmas

Otra forma de clasificación más reciente:

- Dirigido por eventos. El flujo del programa está determinado por sucesos externos (por ejemplo, una acción del usuario).
- Orientado a aspectos. Apunta a dividir el programa en módulos independientes, cada uno con un [Video explicativo](#). comportamiento y responsabilidad bien definido.
- Programación Reactiva
Relacionada con el tratamiento de flujo de datos ya que en principio busca reaccionar ante un evento de forma asincrónica.



Programación Orientada a Objetos

Programación Orientada a Objetos

“Un programa escrito con una lenguaje OO es un conjunto de OBJETOS que **INTERACTÚAN** mandándose **MENSAJES**”

Los elementos que intervienen en la programación OO son:

- Objetos
- Mensajes
- Métodos
- Clases

Objetos:

- Son entidades que poseen estado interno y comportamiento
- Es el equivalente a un dato abstracto

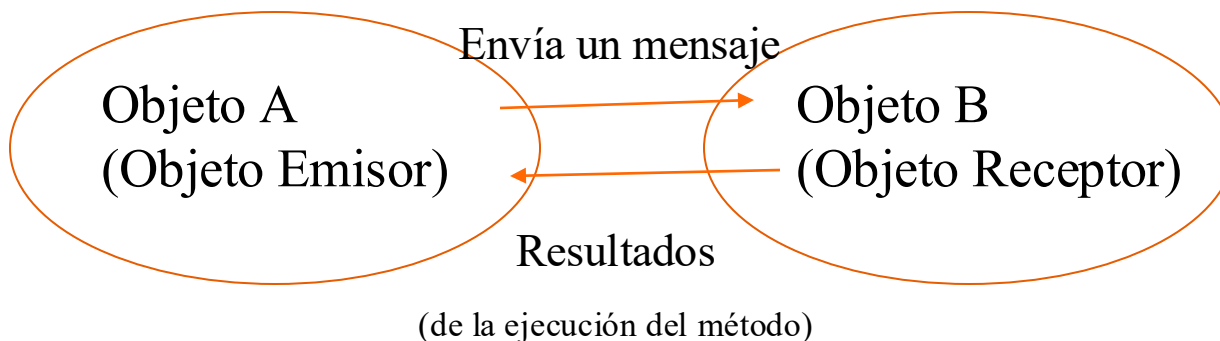
Programación Orientada a Objetos

Mensajes:

- Es una petición de un objeto a otro para que este se comporte de una determinada manera, ejecutando uno de sus métodos
- TODO el procesamiento en este modelo es activado por mensajes entre objetos.

Métodos:

- Es un programa que está asociado a un objeto determinado y cuya ejecución solo puede desencadenarse a través de un mensaje recibido por éste o por sus descendientes



Ejecuta el método correspondiente

Programación Orientada a Objetos

Clases:

- Es un tipo definido por el usuario que determina las estructuras de datos y las operaciones asociadas con ese tipo
- Cada objeto pertenece a una clase y recibe de ella su funcionalidad
- Primer nivel de abstracción de datos: definimos estructura, comportamiento y tenemos ocultamiento.
- La información contenida en el objeto solo puede ser accedida por la ejecución de los métodos correspondientes

Instancia de clase:

- Cada vez que se construye un objeto se está creando una INSTANCIA de esa clase
- Una instancia es un objeto individualizado por los valores que tomen sus atributos

Programación Orientada a Objetos

Otro aspecto de las abstracciones de datos

GENERALIZACIÓN/ESPECIFICACIÓN



HERENCIA

El segundo nivel de abstracción consiste en agrupar las clases en jerarquías de clases (definiendo SUB y SUPER clases), de forma tal que una clase A herede todas las propiedades de su superclase B (suponiendo que tiene una)

Ejemplo: Se tiene definido la siguiente clase

PERSONA

- Nombre
- Edad
- Sexo
- Documento
- Dirección
- Teléfono

- ver-nombre**
- ver-edad**
- ver-teléfono**
- ver-documento**
- ver-sexo**
- cambiar-dirección**
- sacar-documento, etc.**

EMPLEADO

- Curriculum
- cuil

Programación Orientada a Objetos

Otros conceptos adicionales

Polimorfismo:

- Es la capacidad que tienen los objetos de distintas clases de responder a mensajes con el mismo nombre

Ejemplo:

$3 + 5$ Se aplica suma entre
números

“Buenos” + “días” Se concatenan
strings

Binding dinámico:

Es la vinculación en el proceso de ejecución de los
objetos con los mensajes

Programación Orientada a Objetos

C++ (Lenguaje híbrido) Algunas características

- Lenguaje extendido del lenguaje C
- Incorporó características de POO

Los objetos en C++:

- Se agrupan en tipos denominados clases
- Contienen datos internos que definen su estado interno
- Soportan ocultamiento de datos
- Los métodos son los que definen su comportamiento
- Pueden heredar propiedades de otros objetos
- Pueden comunicarse con otros objetos enviándose mensajes

Programación Lógica

Programación Lógica

- La programación lógica es un tipo de paradigmas de programación dentro del paradigma de programación declarativa.
- Es un paradigma en el cual los programas son una serie de aserciones lógicas.
- El conocimiento se representa a través de reglas y hechos.
- Los objetos son representados por términos, los cuales contienen constantes y variables.
- PROLOG es el lenguaje lógico más utilizado.

Elementos de la Programación Lógica

La sintaxis básica es el “**término**”.

- **Variables:**

Se refieren a elementos indeterminados que pueden sustituirse por cualquier otro.

- “*humano(X)*”, la X puede ser sustituida por constantes como: juan, pepe, etc.
- Los nombres de las variables comienzan con mayúsculas y pueden incluir números.

- **Constantes:**

A diferencia de las variables son elementos determinados.

“*humano(juan)*”

Las constantes son string de letras en minúsculas (representan objetos atómicos) o string de dígitos (representan números).

Elementos de la Programación Lógica

Término compuesto:

- Consisten en un "functor" seguido de un número fijo de argumentos encerrados entre paréntesis, los cuales son a su vez términos.
- Se denomina "aridad" al número de argumentos.
- Se denomina "estructura" (ground term) a un término compuesto ***cuyos argumentos no son variables.***

Ejemplos:

padre	→	constante
Longitud	→	variable
tamaño(4,5)	→	estructura

Un functor es una estructura sintáctica utilizada para agrupar términos y formar abstracciones más complejas o estructuradas.

Elementos de la programación Lógica

Listas:

- La constante `[]` representa una lista vacía
- El functor “.” construye una lista de un elemento y una lista. Ejemplo: `.(alpha,[])`, representa una lista que contiene un único elemento que es `alpha`.
- Otra manera de representar la lista es usando `[]` en lugar de `.( )`. Ejemplo anterior la lista quedaría: `[alpha,[]]`
- Y también se representa utilizando el símbolo `|`
`[alpha | []]`

La notación general para denotar lista es : `[X | Y]`
`X` es el elemento cabeza de la lista e `Y` es una lista, que representa la cola de la lista que se está modelando

Cláusula de Horn

- Un programa escrito en un lenguaje lógico es una secuencia de “cláusulas”.
- Las cláusulas pueden ser: un “Hecho” o una “Regla”.

Hecho:

- Expresan relaciones entre objetos
- Expresan verdades
- Son expresiones del tipo $p(t_1, t_2, \dots, t_n)$

Ejemplos:

- `tiene(coche,ruedas)` ☐ representa el hecho que un coche tiene ruedas
- `longuitud([],0)` ☐ representa el hecho que una lista vacía tiene longitud cero
- `moneda(peso)` ☐ representa el hecho que peso es una moneda.

Cláusula de Horn

Regla:

- Cláusula de Horn
- Tiene la forma: conclusión :- condición. (‘:-’ sintaxis prolog)

Dónde:

- :- indica “Si”
 - conclusión es un simple predicado y
 - condición es una conjunción de predicados, separados por comas. Representan un AND lógico
-
- En un lenguaje procedural una regla la podríamos representar como: if condición else conclusión.

Programas y Queries

Ejemplo de programa:

OR { longitud ([],0). **HECHO**
longitud ([X | Y],N) :- longitud(Y, M), N $\neq$ M + 1. **AND**

Programa: conjunto de cláusulas

REGLA

?-longitud([rojo | [verde | [azul | []]]],X).

QUERY

Query: Representa lo que deseamos que sea contestado

Programas y Queries

Programa:

longitud ([],0).

longitud ([X | Y],N) :- longitud(Y, M), N=M + 1.

?-longitud([rojo | [verde | [azul | []]]],X).

longitud([],T) T=0

longitud([azul | []],Z) Z=T+1 => Z=1

longitud([verde | [azul | []]],M) M=Z+1 => M=2

longitud([rojo | [verde | [azul | []]]],X) X=M+1 => X=3

Ejecución de Programas

- Un programa es un conjunto de reglas y hechos que proveen una especificación declarativa de que es lo que se conoce y la pregunta es el objetivo que queremos alcanzar.
- La ejecución de dicho programa será el intento de obtener una respuesta.
- Desde un punto de vista lógico la respuesta a esa pregunta es “YES”, si la pregunta puede ser derivada aplicando “deducciones” del conjunto de reglas y hechos dados.

Ejecución de Programas: Ejemplo

Programa que describe una relación binaria (rel) y su cierre (clos):

rel(a,b). → PROGRAMA

rel(a,c).

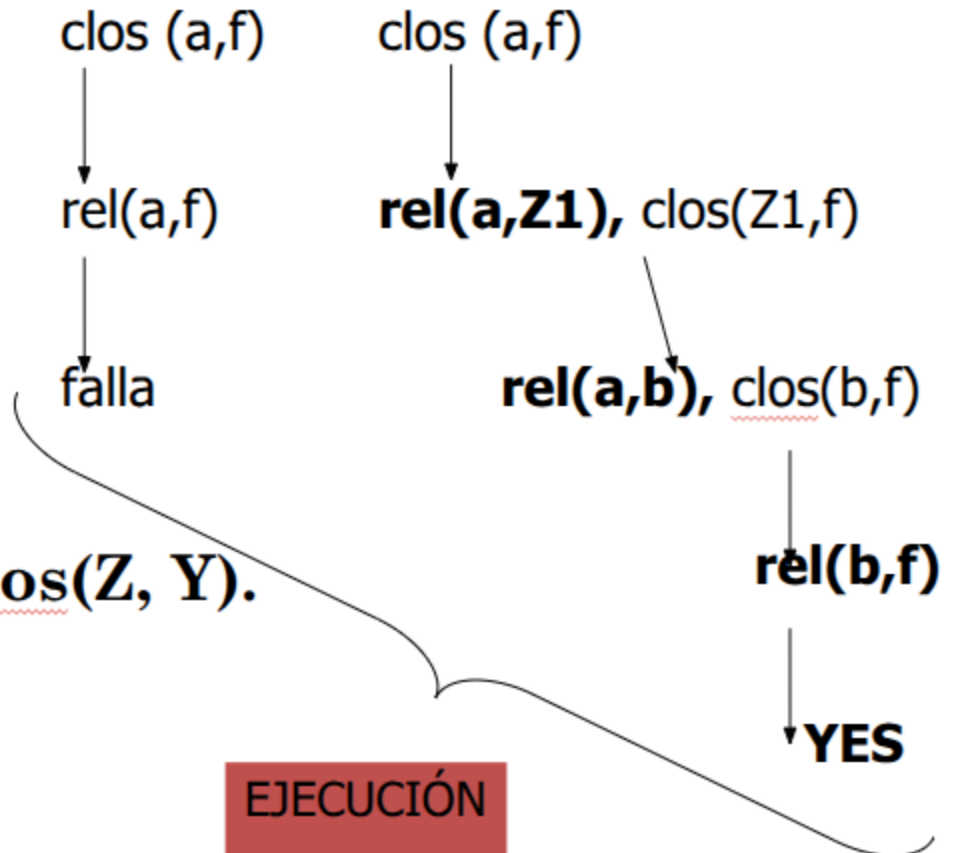
rel(b,f).

rel(f,g).

clos(X,Y) :- rel(X, Y).

clos(X,Y) :- rel(X, Z), clos(Z, Y).

?-clos(a,f) → QUERY



Programación Funcional

Paradigma Aplicativo o Funcional

Basado en el uso de funciones. Muy popular en la resolución de problemas de inteligencia artificial, matemática, lógica, procesamiento paralelo

Ventajas:

- Vista uniforme de programa y función
- Tratamiento de funciones como datos
- Liberación de efectos colaterales
- Manejo automático de memoria

Desventaja:

- Suelen ser interpretados por lo que pueden presentar cierta ineficiencia de ejecución.

Paradigma Funcional

Características de los lenguajes funcionales

- Provee un conjunto de funciones primitivas
- Provee un conjunto de formas funcionales
- Semántica basada en valores
- Transparencia referencial
- Regla de mapeo basada en combinación o composición
- Las funciones de primer orden

Paradigma Funcional

Funciones

- El VALOR más importante en la programación funcional es el de una FUNCIÓN
- Matemáticamente una función es un correspondencia :
 $f: A \rightarrow B$
- A cada elemento de A le corresponde un único elemento en B
- $f(x)$ denota el resultado de la aplicación de f a x
- Las funciones son tratadas como valores, pueden ser pasadas como parámetros, retornar resultados, etc.

Paradigma Funcional

Definiendo Funciones:

- Se debe distinguir entre el VALOR y la DEFINICIÓN de una función.
- Existen muchas maneras de DEFINIR una misma función, pero siempre dará el mismo valor, ejemplo:

DOBLE $X = X + X$

DOBLE' $X = 2 * X$

Denotan la misma función pero son dos formas distintas de definirlas

Tipo de una función

- Puede estar definida explícitamente dentro del SCRIPT, por ejemplo:

cuadrado::num $\rightarrow$ num (define el tipo)

cuadrado $x = x * x$ (definición de la función)

- O puede **deducirse/inferirse** el tipo de una función

Paradigma Funcional

Expresiones y valores

- La expresión es la noción central de la programación Funcional
- Característica más importante:
 - “Una expresión es su VALOR”
- El valor de una expresión depende ÚNICAMENTE de los valores de las sub expresiones que la componen.
- Las expresiones también pueden contener VARIABLES, (valores desconocidos)

Paradigma Funcional

Expresiones y valores

- La noción de Variable es la de “variable matemática”, no la de celda de memoria.
- Las expresiones cumplen con la propiedad de
“TRANSPARENCIA REFERENCIAL”: Dos expresiones sintácticamente iguales darán el mismo valor.
- **No** existen EFECTOS LATERALES”

Paradigma Funcional

- Ejemplos de expresiones para evaluar

Expresión	Valor	
47	47	
(* 4 7)	28	Se está utilizando una Función primitiva
(+ 49 5)	54	

- Definiendo funciones....

cuadrado $x = x * x$

min $x \ y = x$, if $x < y$
 y , if $x > y$

cube $(x) = x * x * x$

Paradigma Funcional

Un script es una lista de definiciones y

- Pueden someterse a evaluación. Ejemplos:

?cuadrado (3 + 4)

49

?min 3 4

3

- Pueden combinarse, Ejemplo:

?min(cuadrado (1 + 1) 3)

3

- Pueden modificarse, ejemplo: Al script anterior le agrego nuevas definiciones:

lado = 12

area = cuadrado lado

Paradigma Funcional

- Algunas expresiones pueden **NO** llegar a reducirse del todo, ejemplo: $1/0$
- A esas expresiones se las denominan **CANÓNICAS**, pero se les asigna un **VALOR INDEFINIDO** y corresponde al símbolo $\text{bottom}(\wedge)$
- Por lo tanto toda **EXPRESIÓN** siempre denota un **VALOR**

Paradigma Funcional

Evaluación de las expresiones:

- La forma de evaluar es a través de un mecanismo de REDUCCIÓN o SIMPLIFICACIÓN
- Ejemplo:

cuadrado (3 + 4)

=> cuadrado 7

se aplicó (+)

=> 7 * 7 (cuadrado)

se aplicó cuadrado

=> 49

se aplicó cuadrado(*)

Otra forma sería:

cuadrado (3 + 4)

=> (3 + 4) * (3 + 4) (cuadrado)

=> 7 * (3 + 4) (+)

=> 7 * 7 (+)

=> 49 (*)

“No importa la forma de evaluarla, siempre el resultado final será el mismo”

Paradigma Funcional

Existen dos formas de reducción:

- Orden aplicativo

Aunque no lo necesite
SIEMPRE evalúa los
argumentos

- Orden normal (lazy evaluation)

- Diferida (Haskell)

No calcula más de lo necesario
La expresión **NO** es evaluada hasta que
su valor se necesite
Una expresión compartida **NO** es
evaluada más de una vez

Paradigma Funcional

TIPOS {
Básicos
Derivados

Básicos: Son los primitivos, ejemplo:
NUM (INT y FLOAT) (Números)
BOOL(Valores de verdad)
CHAR(Characteres)

Derivados: Se construyen de otros tipos, ejemplo:
(num,char) Tipo de pares de valores
(num→→char) Tipo de una función

TODA FUNCIÓN TIENE ASOCIADO UN TIPO

Paradigma Funcional

- Expresiones de tipo polimórficas:

En algunas funciones no es tan fácil deducir su tipo.

Ejemplo:

id x = x

Esta función es la **función Identidad**

Su tipo puede ser de $\text{char} \rightarrow \text{char}$, de $\text{num} \rightarrow \text{num}$, etc.

Por lo tanto su tipo será de $\beta \rightarrow \beta$ (denota un tipo polimórfico)

Se utilizan **letras griegas** para representar **tipos polimórficos**

Otro ejemplo: **letra x = “A”**

Su tipo será $\beta \rightarrow \text{char}$

Paradigma Funcional

- Currificación:

- Mecanismo que reemplaza argumentos estructurados por argumentos más simples.
- Ejemplo: sean dos definiciones de la Función “Suma”

1. $\text{Suma}(x,y) = x + y$

2. $\text{Suma}' x y = x + y \quad \longrightarrow \quad \text{Suma}' x \ y = \text{Suma}'x (y) = x + y$

Existen entre estas dos definiciones una diferencia sutil:
“Diferencia de tipos de función”

El tipo de Suma es : $(\text{num}, \text{num}) \rightarrow \square \text{ num}$

El tipo de Suma' es : $\text{num} \rightarrow (\text{num} \rightarrow \square \text{ num}) \leftarrow$

Aplicando la función:

$$\text{Suma}(1,2) \rightarrow 3$$

$$\text{Suma}' 1 \ 2 \ \square \quad \text{Suma}'1 \text{ aplicado al valor} \\ 2 \rightarrow 3$$

Por cada valor de
x devuelve una
función

Para todo los
valores devuelve el
siguiente

Paradigma Funcional

Cálculo Lambda

- El un modelo de computación para definir funciones
- Se utiliza para entender los elementos de la programación funcional y la semántica subyacente, independientemente de los detalles sintácticos de un lenguaje de programación en particular.
- Es un modelo de programación funcional que se independiza de la sintaxis del lenguaje de programación



Paradigma Funcional

Las expresiones del Lambda cálculo pueden ser de 3 clases:

- Un simple identificador o una constante. Ej: x , 3
- Una definición de una función. Ej: $\lambda x.x+1$

Una aplicación de una función. La forma es $(e1\ e2)$, dónde se lee $e1$ se aplica a $e2$.

Ej: en la funcion cube $(x) = x * x * x$

$\lambda x. x * x * x$

$\lambda (x. x * x * x) 2$

Evaluamos la
función con 2
y resulta en 8

Enlaces

[Programación Lógica](#)

[Programación Funcional](#)

[Programación Funcional \(video\)](#)

Programación Orientada a Aspectos

Programación orientada a Aspectos

Divide responsabilidades en módulos conocidos como **aspectos**.

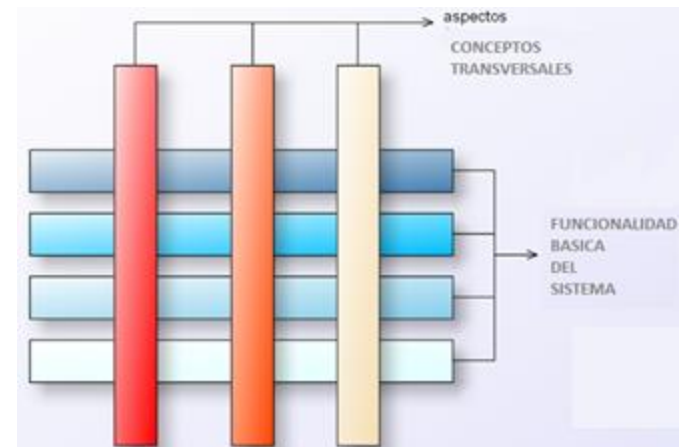
Estos aspectos suelen ser “transversales” a toda la aplicación.

Se puede combinar con otros paradigmas.

Su implementación está en un solo lugar en vez de repartirse en todo el código. (Mantenimiento y modificabilidad)

Usos:

- Autenticación
- Seguridad
- Trazabilidad
- Control de errores
- Otros.



Programación orientada a Aspectos

Conceptos

- Aspecto: Funcionalidad transversal de la aplicación (Ej. login, logs del sistema)
- Punto de enlace (joinpoint): Punto del programa donde el aspecto es conectado.
 - Llamada a un método
 - lanzamiento de excepción
 - Modificación de campo
 - Etc.
- Consejo (advice): Contiene el código que se ejecutará al implementar el Aspecto. Se inserta a través de los joinpoints.
- Punto de corte: define que advice (consejo) se usa en cada joinpoint.
- Introducción: Permite añadir métodos o atributos a una clase existente. Ej: en una auditoría podríamos establecer la fecha de modificación (como atributo) de un objeto.
- Target (objetivo): Clase sobre la que se aplica el aspecto.
- Proxy: Objeto resultante de aplicar un advice (consejo) a un objeto.
- Weaving (costura): integrar el advice (consejo) a los Target de nuestra aplicación.

Programación Orientada a Aspectos

Formas de definir cuando se ejecuta un aspecto.

- Antes
- Antes de retornar luego de la ejecución de un método.
- Después de una excepción
- Otras

Desventajas

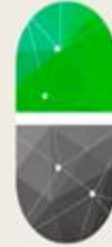
- Puede ser engorroso de seguir el flujo del programa (tanto en debug como al depurar)
- Dificultad de identificación de problemas.

Ventajas

- Mejora de modularidad
- Elimina dependencias
- Facilita la extensibilidad
- Reutilización de código
- Mejora el mantenimiento del código
- Código más limpio

Programación Orientada a Aspectos

AOP



```
public void insertarCliente (Cliente elCliente, String IdUsuario){
```

```
// código para login
```

```
// código para comprobación usuario válido
```

```
Session miSesion=sessionFactory.getCurrentSession();
```

```
miSesion.save (elCliente);
```

```
}
```

Controlador

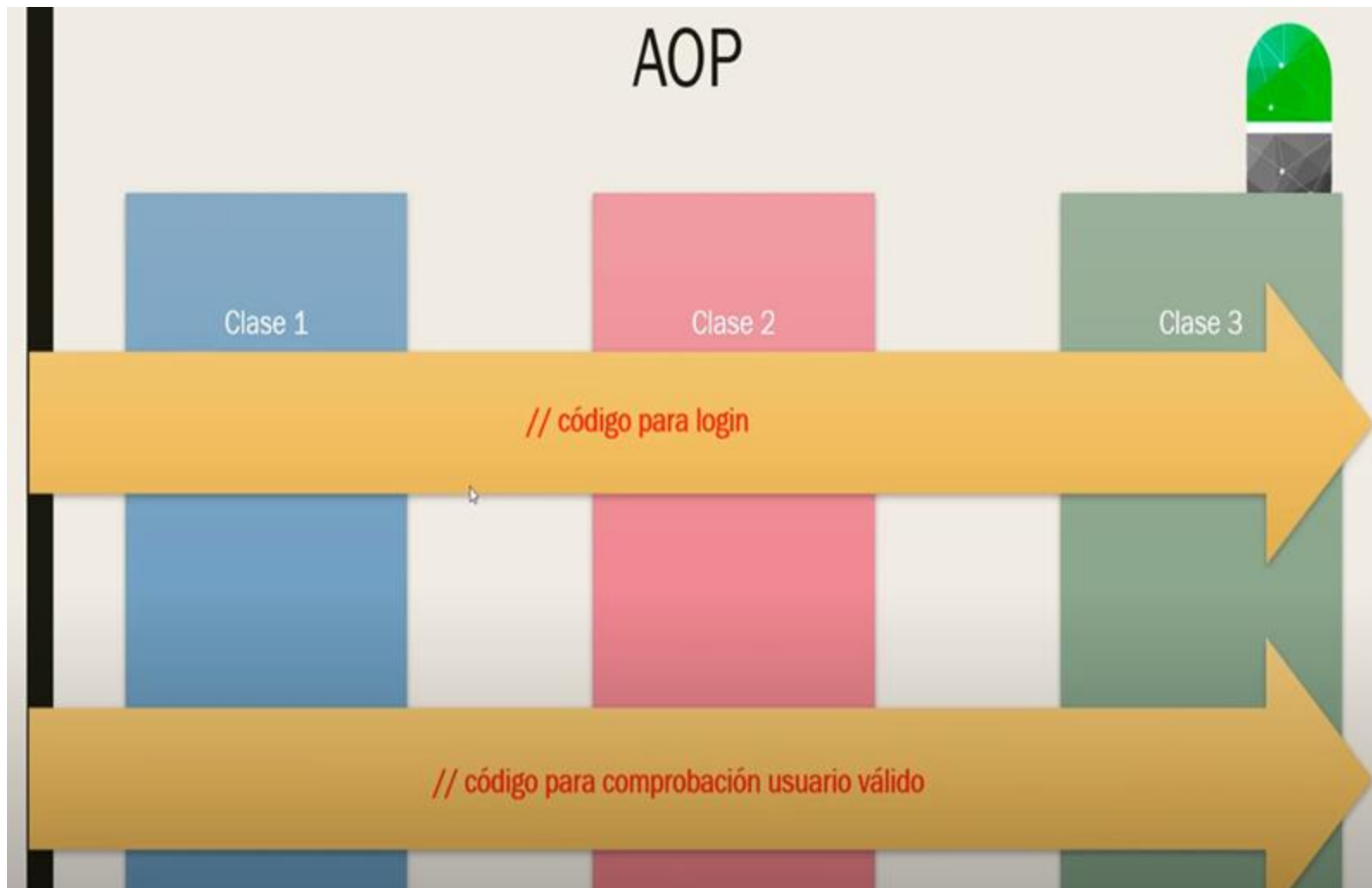
```
// código para login *  
// código para comprobación
```

ClienteDAO

```
// código para login  
// código para comprobación  
usuario válido
```



Programación orientada a Aspectos



Programación Orientada a Aspectos

```
public class Hello {  
  
    public static void main(String[] args) {  
        sayHello();  
    }  
  
    public static void sayHello() {  
        System.out.print("Hola ");  
    }  
}
```

```
public aspect AspectHello {  
  
    pointcut greeting() : execution (* Hello.sayHello(..));  
  
    before(): greeting() {  
        System.out.println("Bienvenidos a POA");  
    }  
  
    after() returning() : greeting() {  
        System.out.println("clase de lenguajes de programacion 2022-1");  
    }  
}
```

Bienvenidos a POA

Hola clase de lenguajes de programacion 2022-1

Process finished with exit code 0

Programación Orientada a Aspectos

Ejemplos de lenguajes con programación orientada a aspectos:

- Java: A través de frameworks como Spring AOP y AspectJ.
- C#: Con el framework PostSharp, entre otros.
- Python: Con librerías como Python AOP.
- Perl: Mediante el módulo Aspect.
- Ruby: Con librerías como Ruby AOP.

Programación Reactiva

Programación Reactiva

La Programación Reactiva es un paradigma declarativo relacionada con el tratamiento de flujo de datos ya que en principio busca **reaccionar** ante un **evento** de forma **asíncronica**. "Asíncrono" significa que algo no sucede en el mismo momento que otro evento, o que no está sincronizado con él. Tiene mucho que ver con el concepto de microservicios y se basa en un [Manifiesto](#).

- Responsivos: Capacidad de completar la tarea en un tiempo determinado.
- Resilientes: Soportar y recuperarse ante errores.
- Elásticos: Pueden aumentar o disminuir los recursos para adaptarse a cambios.
- Orientados a Mensajes: Basado en un sistema de intercambio de mensajes de forma asíncrona entre componentes.

Programación Reactiva

Está basado en el patron “**observer**”, que define una relación del tipo (uno a muchos) entre objetos. Cuando el estado de un objeto cambia, este notifica el cambio al resto de los objetos dependientes. La idea básica es que un objeto contiene atributos o métodos observables que otros objetos pueden suscribir pasando una referencia a sí mismos. El objeto observable mantiene una lista de quienes lo observan y le notifica los cambios.

Componentes

- Observable: Interface que implementa todos los objetos observados.
- Observador: Interface que implementa los objetos que desean observar.
- Suscripción: Función que permite suscribir o desuscribir al observable.

Técnicas de Programación: Programación Asíncrona

La programación asíncrona no es un paradigma de programación sino una técnica para resolver determinados problemas que requieren una solución en donde por algún motivo:

- Peticiones HTTP (APIs, microservicios)
- Acceso a bases de datos
- Lectura/escritura de archivos
- Timers, colas de mensajes, eventos
- Etc.

la ejecución del programa tiene que esperar por algo externo y quizá lento. Por lo tanto, en estos casos, bloquear el programa mientras espera es un desperdicio de tiempo y recursos. La asincronía permite que el programa siga haciendo otras tareas mientras espera el resultado.

Técnicas de Programación: Programación Asíncrona

La programación asíncrona permite ejecutar tareas largas (como peticiones de red o lectura de bases de datos) sin bloquear el hilo principal, mejorando drásticamente la capacidad de respuesta de la aplicación y la escalabilidad del servidor.

Cuando usarlo:

- Hay múltiples operaciones I/O independientes
- Necesitas alta concurrencia
- Trabajas con APIs externas
- Construyes servidores web

Evitar cuando:

- Solo hay operaciones CPU-intensivas (sin I/O)
- El código es simple y secuencial

Técnicas de Programación:

Programación Asíncrona

```
1 const esperar = function (ms) {
2   return new Promise(resolve => setTimeout(resolve, ms));
3 }
4
5 async function hacerDesayuno() {
6   console.log("hd1. Poner agua a hervir...");
7   await esperar(4000);           // ← espera 4s sin bloquear
8   console.log("hd2. Agua lista → hacer té");
9
10  console.log("hd3. Poner pan en tostadora...");
11  // asigna la promesa, la tostadora ya está corriendo en segundo plano
12  const tostadaLista = esperar(3000);
13  // mientras tanto hago otra cosa (ej. calentar leche/agua)
14  await esperar(3000);
15  console.log("hd4. Echando café instantáneo...");
16  // ahora sí espero que termine la promesa de la tostada
17  await tostadaLista;
18  console.log("hd5. Untando mantequilla y mermelada");
19
20  return "¡DESAYUNO LISTO!";
21 }
22
23 async function prepararComputadora() {
24   console.log("pc1. Encender Computadora...");
25   await esperar(4000);           // ← espera 4s sin bloquear
26   console.log("pc2. Abrir Navegador");
27   return "COMPUTADORA LISTA!";
28 }
29
30 console.log("Inicio del día");
31 hacerDesayuno()
32   .then(resultado => console.log(resultado))
33   .catch(err => console.error("Algo salió mal", err));
34
35 console.log("Mientras tanto... reviso el móvil, contesto mensajes");
36
37 prepararComputadora()
38   .then(resultado => console.log(resultado))
39   .catch(err => console.error("Algo salió mal", err));
40
41 console.log("Abro la ventana para ver el clima");
42
```

Inicio del día

hd1. Poner agua a hervir...

Mientras tanto... reviso el móvil,

contesto mensajes

pc1. Encender Computadora...

Abro la ventana para ver el clima

hd2. Agua lista → hacer té

hd3. Poner pan en tostadora...

pc2. Abrir Navegador

COMPUTADORA LISTA!

hd4. Echando café instantáneo...

hd5. Untando mantequilla y mermelada

¡DESAYUNO LISTO!

Con la sintaxis “async/await” logramos que la unidad no bloquee el hilo principal demorandose y continúa con la ejecución. Luego de terminar la “ejecución asíncrona” continua la ejecución.

Una **Promesa** es un objeto que representa la terminación o el fracaso eventual de una operación asíncrona (como consultas a APIs o lectura de archivos) sin bloquear la ejecución del hilo principal.

Lenguajes Basados en Script

Lenguajes Basados en Script - Situación lenguajes previos

Los lenguajes de programación tradicionales están destinados principalmente para la construcción de aplicaciones auto-contenidas:

- Lenguajes como C++ y Ada abordan el problema de desarrollar grandes aplicaciones desde cero empleando equipos de programadores profesionales, partiendo de especificaciones bien definidas y cumpliendo con restricciones de rendimiento específicas. En general se generaban programas que aceptan una suerte entrada, la procesan de una manera bien entendida y finalmente generan una salida apropiada.
- Tienden a mejorar eficiencia, mantenibilidad, portabilidad y detección estática de errores. Los tipos se construyen alrededor de conceptos a nivel hardware como enteros de tamaño fijo, punto flotante, caracteres y arreglos. Esto requiere un diseño e implementación del lenguaje que sea robusto y eficiente.
- Sin embargo, muchos de los usos en diferentes entornos, requerían la coordinación de múltiples programas, por lo que podría ser necesario “combinar” o “pegar” estos programas de alguna forma. Y en este caso se buscaba la forma más sencilla y directa de lograrlo.

Lenguajes Basados en Script - Orígenes y Definición

El uso de la palabra «script» se remonta a principios de la década de 1970, cuando los creadores del sistema operativo UNIX acuñaron el término «shell script» para referirse a una secuencia de comandos que debían leerse desde un archivo y ejecutarse en orden. El nombre script se utiliza para un archivo de texto destinado a ser ejecutado directamente, en lugar de ser compilado en un formato de archivo diferente antes de su ejecución.

En un principio los lenguajes de scripting eran un conjunto de comandos escritos en un archivo para ser interpretado. A este archivo se lo denomina “script”. Los primeros LBS eran un conjunto de comandos que eran interpretados como llamadas al sistema como manejo o filtrado de archivos.

Luego se agregaron variables, flujo de control, etc. y fueron escalando hasta convertirse en lenguajes de programación completos como los conocidos actualmente. (Perl, Python, PHP, javascript, etc.)

Lenguajes Basados en Script

“Los lenguajes script asumen la existencia de componentes útiles en otros lenguajes. Su intención no es escribir aplicaciones desde el comienzo sino por combinación de componentes”

John Ousterhout

Creador de TCL

“Tool Command Language”

Lenguajes Basados en Script - Definición

Hoy en día, el término «scripting» se utiliza con tres significados distintos:

- Un nuevo estilo de programación: Este permite que las aplicaciones se desarrollen de manera mucho más rápida que con los métodos tradicionales, y hace posible que evolucionen velozmente para satisfacer los requisitos cambiantes del usuario. Las aplicaciones construidas de esta manera se denominan a veces «glue applications» (aplicaciones de pegamento), y el lenguaje utilizado se conoce como «glue language» (lenguaje de pegamento).
- Manipular, personalizar y automatizar las funciones de un sistema existente: Tal como lo establece la definición de ECMAScript (el estándar detrás de JavaScript). En este caso, el script se utiliza para controlar una aplicación que proporciona una interfaz programable. Ejemplos de esto son el uso de scripting del lado del cliente (client-side scripting) y el HTML Dinámico para crear páginas web interactivas.
- Como alternativa a un lenguaje convencional para tareas de programación general: particularmente en la programación de sistemas y la administración de sistemas. Los administradores de sistemas UNIX han utilizado durante mucho tiempo lenguajes de scripting para tareas de mantenimiento del sistema.

Lenguajes Basados en Script - Objetivos

- Uso de scripts para “pegar” o combinar programas. Los lenguajes script de propósito general (Perl, Python) suelen conocerse como Glue-languages. Se diseñaron para “pegar” programas existentes a fin de construir un sistema mas grande. Se utilizan como lenguajes de extensión, ya que permiten al usuario adaptar o extender las funcionalidad de las herramientas script.
- Administración de sistemas: automatización de tareas cotidianas y construcción de herramientas de reducción de datos (data reduction tools). El concepto de scripting surge inicialmente de las necesidades de la administración de sistemas en el mundo UNIX. Los administradores utilizaban shell scripts (secuencias de comandos de la shell ejecutados desde un archivo) para automatizar tareas cotidianas. Con la evolución de los LBS, se pudieron lograr efectos sofisticados, convirtiendo a los scripts en alternativas viables a la programación en C para la mayoría de las actividades de este tipo.

Lenguajes Basados en Script - Objetivos

- Control remoto de aplicaciones. La automatización de aplicaciones es cada vez más importante y puede aplicarse a cualquier aplicación scriptable, es decir, una que posea algún tipo de interfaz de control que permita ser manejada o utilizada por otro programa.
- Extensiones de sistemas y aplicaciones: Muchas aplicaciones modernas exponen una API programable junto con un lenguaje de scripting —denominado aún lenguaje de macros, pero que es estrictamente un lenguaje de extensión— para ampliar la interfaz de usuario. (Para un usuario común, una macro parece ser simplemente una forma de grabar selecciones de menú y pulsaciones de teclas para automatizar operaciones frecuentes; pero lo que ocurre en realidad es que un «compilador de pulsaciones» está generando un programa en el lenguaje de macros subyacente).
- Construcción de interfaces de línea de comandos (CLI): El scripting visual es el proceso de construir una interfaz gráfica a partir de una colección de objetos visuales («controles» en la terminología de Microsoft, «widgets» en Tcl/Tk y Perl-Tk), los cuales tienen propiedades (atributos), como colores de primer plano y fondo, texto en un botón, etc., que pueden ser configurados por un programa escrito en un lenguaje apropiado. Algunos objetos (por ejemplo, los botones) responden a eventos externos como clics del ratón, y la respuesta a cada acción se define mediante un script. La programación de dicha interfaz se describe como «scriptear los objetos».

Lenguajes Basados en Script - Objetivos

- Programación "experimental". Los lenguajes tradicionales como C se desarrollaron como herramientas para escribir programas grandes y complejos, muchos de los cuales requieren equipos enteros de programadores. Sin embargo, existe otra clase de programas cuya escritura implica experimentación, ya que los requisitos precisos no están claros desde el principio y a menudo es necesario producir varias versiones antes de que la herramienta sea satisfactoria. Los lenguajes de scripting son ideales para este tipo de «programación experimental», dado que las ideas pueden probarse sin incurrir en la penalización de tiempo que supone la sobrecarga del ciclo de editar-compilar-enlazar-cargar.
- Procesamiento de formularios del lado del servidor: en la Web mediante el uso de CGI. Desde sus inicios, el HTML ha permitido un tipo limitado de interacción con el usuario mediante el uso de formularios: una interfaz visual bastante primitiva, definida por etiquetas en un documento HTML. En la implementación original de la Web, cuando se envía un formulario para su procesamiento, la información introducida por el usuario se codifica y se envía al servidor para ser procesada por un script CGI, que finalmente genera una página HTML para ser enviada de vuelta al navegador. Este procesamiento requiere la manipulación de cadenas (strings) para decodificar los datos del formulario y la manipulación de texto para construir la página HTML que constituye la respuesta; también puede requerir acceso al sistema para ejecutar otros procesos. Como alternativa a realizar todo el procesamiento de un formulario con un script que se ejecuta en el servidor, es posible realizar cierto procesamiento en el lado del cliente (client-side) dentro del propio navegador para validar los datos del formulario antes de enviarlos al servidor, utilizando un lenguaje de scripting diseñado para tal fin, como JavaScript/JScript o VBScript.

Lenguajes Basados en Script - Características

- Asociado a editores livianos: La asociación entre los lenguajes de scripting y los editores de texto livianos (como Vim, notepad++, etc.) Como el intérprete se encarga de todo "detrás de escena", el programador solo necesita un sitio donde escribir texto plano. Un editor liviano es suficiente porque el "trabajo pesado" lo hace el motor del lenguaje al momento de correr el archivo.

Lenguajes Basados en Script - Características

- Interpretados – (modestos requerimientos de eficiencia): Compilación y ejecución integradas. Esta es, quizás, la característica más importante. Los lenguajes de scripting suelen catalogarse como lenguajes interpretados, pero esto es una simplificación excesiva. Lo que realmente importa es que se comportan como si fueran interpretados; es decir, operan bajo una base de ejecución inmediata, sin la necesidad de emitir comandos separados para compilar el programa y luego ejecutar el archivo objeto resultante, y sin necesidad de enlazar extensas librerías al código objeto. Esta noción de ejecución inmediata (a veces descrita como "hack-it-and-run" o "pícalo y ejecútalo") es vital, porque el scripting es a menudo una actividad interactiva y experimental que no encaja bien con el ciclo de editar-compile-enlazar-ejecutar de la programación convencional. La eficiencia no es un problema. La facilidad de uso se logra a expensas de la eficiencia.

Lenguajes Basados en Script - Características

- Alto nivel de funcionalidad en aplicaciones de áreas específicas: Los lenguajes de scripting suelen tener una funcionalidad mejorada en áreas específicas. Por ejemplo, la mayoría de los lenguajes proporcionan una potente manipulación de cadenas basada en el uso de expresiones regulares, mientras que otros ofrecen un acceso sencillo a funciones de bajo nivel del sistema operativo, o a la API o el modelo de objetos exportado por una aplicación. Ejemplos son: Lenguaje de comandos (shell), Procesamiento de texto y Generación de Reportes

Lenguajes Basados en Script - Características

Tipado Dinámico:

Cuando se utilizan como glue-languages puede necesitarse intercambiar datos de distinto tipo entre distintos subsistemas y estos pueden ser incompatibles. Por esto, si el LBS tiene un sistema de tipos simple podría ser demasiado inflexible y por otro lado uno muy completo atendería contra un rápido desarrollo y evolución del sistema.

Por lo general los tipos no necesitan ser declarados (aunque esto los haga más difíciles de leer).

Si bien el tipado dinámico hace a los LBS menos complicados, los hace más inseguros y menos eficientes.

Lenguajes Basados en Script - Características

En los LBS las declaraciones son escasas o nulas y proveen reglas simples que gobiernan el alcance de los identificadores.

Por ejemplo, en Perl, cada identificador es global por omisión (hay opciones para limitar el alcance).

En otros lenguajes (e.g., PHP y Tcl), cada cosa es local por omisión (un objeto global debe ser explícitamente importado).

Python adopta la regla: “a una variable que se le asigna un valor es local al bloque donde aparece dicha asignación” (se puede cambiar esta regla con una sintaxis especial).

Lenguajes Basados en Script - Características

Dado la falta de declaraciones, muchos LBS incorporan tipificación dinámica.

En algunos lenguajes el tipo de la variable es chequeada inmediatamente antes de su uso: e.g., PHP, Python, Ruby, y Scheme.

En otros, el tipo de una variable (por ende su valor) será interpretado de manera diferente según el contexto: e.g., REXX, Perl, y Tcl.

Lenguajes Basados en Script - Características

Ejemplo en Perl

```
$a = "4";  
print $a . 3 . "\n"; # '.' es la concatenación  
print $a + 3 . "\n"; # '+' es la suma
```

Dará la siguiente salida:

43

7

Lenguajes Basados en Script - Características

Los LBS son ricos en:

- Conjuntos
- Diccionarios
- Listas
- Tuplas, etc.

Por ejemplo:

En muchos LBS es común poder indizar arreglos a través de cadenas de caracteres, lo que implica una implementación de tablas de hash y manejo de almacenamiento usando "garbage collection".

Lenguajes Basados en Script - Usos

Principales Usos:

- Lenguaje de comandos (shell)
- Procesamiento de texto y Generación de Reportes
- Matemática y Estadística
- Lenguajes de "pegado"(GLUE) y de propósito general
- Extensión de Lenguajes
- WWW como ejemplo especial
- CGI (Common Gateway Interface)
- Scripts Embebidos en Servidores
- Scripts Embebidos en Clientes
- Otros: XML.

Lenguajes Basados en Script - Javascript

Javascript fue desarrollado para el navegador Netscape en los años 90.

El lenguaje estándar fue desarrollado a fines de los años 90 por European Computer Manufacturers Association (ECMA) como ECMA-262.

Aunque el intérprete javascript puede ser embebido en cualquier aplicación por lo general se utiliza en los navegadores web, donde el código se asocia a documentos HTML para darles dinamismo y control.

La sintaxis es similar a Java pero por el contrario es dinámicamente tipado.

Los strings y arreglos tienen longitud dinámica y sus índices no son chequeados.

Soporta OO pero no Herencia (al menos en su versión inicial). (Herencia por prototipos)

Uno de los usos más importantes es la manipulación dinámica del documento HTML.

Lenguajes Basados en Script - Python

Python:

- Software Libre.
- Interpretado (intérprete de comandos).
- Multiplataforma.
- Multiparadigma (Imperativo, OO, funcional).
- Fuertemente Tipado.
- Dinámicamente Tipado.
- Sintaxis simple y legible.
- Las variables NO se declaran. Nacen cuando se le asigna un valor. (variables dinámicas)
- Tipos Básicos – Cadenas, listas, tuplas, conjuntos, diccionarios.